Pixel-Perfect Depth Review

Testing Flying-Pixel Reduction via a Simple Heuristic
on Depth Anything v2 Predictions

IASD – Point Clouds and 3D Modeling

Jules Roques

April 2026

Abstract

Pixel-Perfect Depth (PPD) is a recent monocular depth estimation model that operates in pixel space to produce depth maps free of *flying pixels*, artifacts that appear as floating points near object edges when depth maps are lifted to 3D point clouds. In this project we investigate whether a simple, post-hoc heuristic applied to the depth maps of a faster discriminative model (Depth Anything v2) can approximate the edge-quality benefits of PPD. We benchmark three configurations: PPD, Depth Anything v2 (DAv2), and DAv2 with flying-pixel cleaning (DAv2-Cleaned) on the Hypersim test set, measuring inference time, depth F-scores, and an edge-aware Chamfer distance. Results show that the heuristic significantly improves edge quality at a fraction of PPD's inference cost, but comes with a trade-off in point coverage and fails on highly textured scenes such as vegetation. The code is available at `https://github.com/jules-roques/pixel-perfect-depth-review`.

Contents

1 Summary of Pixel-Perfect Depth **2**
1.1 Motivation: the Flying Pixels Problem . . . 2
1.2 Core Idea: Pixel-Space Diffusion . . . 2
1.3 Semantics-Prompted Diffusion Transformers (SP-DiT) . . . 2
1.4 Cascade DiT Design (Cas-DiT) . . . 3
1.5 Generative Formulation: Flow Matching . . . 3
1.6 Training and Depth Normalization . . . 3
1.7 Results Claimed by the Paper . . . 3
1.8 Known Limitations . . . 3

2 Experimental Setup **4**
2.1 Dataset: Hypersim Test Set . . . 4
2.1.1 Pre-processing: Distance-to-Depth Conversion . . . 4
2.2 Models Under Evaluation . . . 4
2.3 Flying-Pixel Cleaning Heuristic . . . 4
2.4 Depth Alignment for Metric Recovery . . . 5

3 Evaluation Metrics **5**
3.1 Depth F-score ($\delta = 1.25$) . . . 5
3.2 Edge-Aware Chamfer Distance . . . 6
3.3 Inference Time . . . 6

4 Benchmark Pipeline **6**

5 Results **7**
5.1 Quantitative Results . . . 7
5.2 Qualitative Results: 3D Point Clouds . . . 7

6 Discussion **8**
6.1 Trade-offs of the Heuristic Approach 8
6.2 End-to-End vs. Post-Hoc 9

7 Conclusion **10**

1 Summary of Pixel-Perfect Depth

1.1 Motivation: the Flying Pixels Problem

Monocular depth estimation (MDE) has made dramatic progress in recent years, with models such as Depth Anything v2 [6], Depth Pro [1], and Marigold [2] achieving impressive zero-shot performance. However, when the estimated depth maps are unprojected into 3D point clouds, a systematic artifact appears at object boundaries: *flying pixels*.

Flying pixels are spurious 3D points that lie between the foreground and background surfaces. They arise from two distinct causes depending on the modelling paradigm:

- **Discriminative models** (e.g. Depth Anything v2, Metric3D) minimize a regression loss and tend to predict an *averaged* depth at depth-discontinuous edges, blending the foreground and background values.
- **Generative (latent-diffusion) models** (e.g. Marigold, GeoWizard, Lotus) bypass the averaging issue by modelling a depth distribution, but they rely on a Variational Autoencoder (VAE) to compress depth maps into a latent space. The VAE's lossy reconstruction introduces blurring at edges and fine structures, re-introducing flying pixels.

The Pixel-Perfect Depth (PPD) paper [5] (NeurIPS 2025) directly addresses this limitation.

1.2 Core Idea: Pixel-Space Diffusion

The key insight of PPD is to perform diffusion generation *directly in the pixel space*, bypassing the VAE entirely. By removing the lossy encoding/decoding bottleneck, the model can in principle produce depth maps whose edges are arbitrarily sharp, yielding flying-pixel-free point clouds.

However, pixel-space diffusion at high resolution (1024×768) is both computationally expensive and notoriously difficult to optimize: the model must simultaneously capture both the global scene layout (low-frequency content) and fine-grained edge details (high-frequency content). The authors find that a naïve pixel-space DiT produces extremely poor depth predictions.

1.3 Semantics-Prompted Diffusion Transformers (SP-DiT)

To overcome the optimization difficulty, PPD introduces **Semantics-Prompted Diffusion Transformers (SP-DiT)**. The idea is to inject high-level semantic representations from a pre-trained vision foundation model (VFM) into the DiT blocks so that the diffusion process is *prompted* to maintain global semantic consistency.

Concretely, given an input image $\mathbf{c}$, a pre-trained encoder f (e.g. Depth Anything v2's ViT-L encoder) extracts semantic features:

$$\mathbf{e} = f(\mathbf{c}) \in \mathbb{R}^{T' \times D'}, \tag{1}$$

where T' is the number of tokens and D' the embedding dimension. These features are L2-normalised along the feature dimension:

$$\hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2}, \tag{2}$$

and then fused into the DiT tokens $\mathbf{z}$ via a learnable MLP h_ϕ after bilinear spatial interpolation $\mathcal{B}$:

$$\mathbf{z}' = \mathbf{z} + h_\phi\big(\mathcal{B}(\hat{\mathbf{e}})\big). \tag{3}$$

The normalization prevents magnitude mismatch between the external semantics and the internal DiT representations, which the authors show is critical for training stability. Four different VFMs (DINOv2, MAE, VGGT, Depth Anything v2) are evaluated; all yield significant improvements, with Depth Anything v2 performing best.

1.4 Cascade DiT Design (Cas-DiT)

To further reduce computational cost, PPD adopts a **Cascade DiT Design**. The architecture consists of $N = 24$ DiT blocks split into two stages:

- **Coarse stage (blocks 1–12):** Standard DiT blocks with a large patch size of 16, producing $(H/16) \times (W/16)$ tokens. This design reduces the token count and encourages the model to focus on modelling global scene structure.
- **Fine stage (blocks 13–24, i.e. SP-DiT):** After an MLP-based channel expansion and reshaping, the tokens are densified to $(H/8) \times (W/8)$. These blocks receive the semantic prompts and are responsible for generating fine-grained spatial details.

This coarse-to-fine cascaded design mirrors the hierarchical nature of visual perception and reduces inference time by approximately 30% compared to a uniform-patch baseline, while simultaneously improving depth accuracy.

1.5 Generative Formulation: Flow Matching

PPD uses **Flow Matching** as its generative backbone. Given a clean depth sample $\mathbf{x}_0 \sim \mathcal{D}$ and Gaussian noise $\mathbf{x}_1 \sim \mathcal{N}(0, I)$, an interpolated sample is defined as:

$$\mathbf{x}_t = (1 - t)\,\mathbf{x}_0 + t\,\mathbf{x}_1, \quad t \in [0, 1], \tag{4}$$

with velocity field $\mathbf{v}_t = \mathbf{x}_1 - \mathbf{x}_0$. The model $\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c})$ is trained to predict this velocity via an MSE loss:

$$\mathcal{L} = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_1} \left\| \mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|^2. \tag{5}$$

At inference, the depth is obtained by integrating the learned ODE from $t = 1$ (noise) to $t = 0$ (depth) using an Euler sampler with a few steps (typically 4–10).

The input to each DiT forward pass is the channel-wise concatenation $\mathbf{a}_t = \mathbf{x}_t \oplus (\mathbf{c} - 0.5)$, where $\mathbf{c}$ is the RGB image.

1.6 Training and Depth Normalization

Ground-truth depth values are first transformed to log scale ($\tilde{d} = \log(d + \epsilon)$, $\epsilon = 1$) for balanced capacity across indoor/outdoor scenes, then min–max normalized using the 2nd and 98th depth percentiles. The model is trained on the Hypersim dataset (54K samples) at 512×512, and additionally on UrbanSyn, UnrealStereo4K, VKITTI, and TartanAir for the 1024×768 variant.

Notably, the architecture is *purely transformer-based* with no convolutional layers, distinguishing it from prior depth estimation models.

1.7 Results Claimed by the Paper

PPD achieves state-of-the-art performance among all published generative models on five benchmarks (NYUv2, KITTI, ETH3D, ScanNet, DIODE), and *significantly* outperforms all other models in the proposed edge-aware point cloud evaluation on Hypersim. The authors additionally introduce an edge-aware Chamfer distance metric that extracts edge masks from ground-truth depth via the Canny operator and measures the Chamfer distance between predicted and ground-truth point clouds near those edges.

1.8 Known Limitations

The authors acknowledge two limitations:

1. **Temporal inconsistency:** like most image-based diffusion models, PPD lacks temporal coherence across video frames.
2. **Slow inference:** the multistep diffusion process makes PPD approximately 8× slower than Depth Anything v2, though comparable to Depth Pro.

2 Experimental Setup

The goal of our project is to test the central contribution of PPD—flying-pixel reduction at edges—by asking a simple question: *can a post-hoc heuristic applied to a fast discriminative model (Depth Anything v2) achieve comparable edge quality?*

2.1 Dataset: Hypersim Test Set

We evaluate on the **Hypersim** dataset [4], a large-scale photorealistic synthetic dataset of indoor scenes. Hypersim provides:

- High-dynamic-range (HDR) photometric RGB images stored as HDF5 files;
- Ground-truth *distances from camera* (Euclidean range depth) as HDF5 files;
- Per-scene camera intrinsics mapping normalized device coordinates (NDC) to camera coordinates.

We use the official **test split**, which was *not* used during training of either PPD or Depth Anything v2. The test set contains more than 7000 images spanning a variety of indoor environments with complex geometry, furniture, staircases, and vegetation. We selected 100 images from this set to perform the benchmark.

2.1.1 Pre-processing: Distance-to-Depth Conversion

Hypersim stores *Euclidean distances from the camera center* rather than planar (z-buffer) depth. To convert to planar depth, we compute the ray direction for each pixel using the camera intrinsics matrix $M_{\text{cam}}^{\text{ndc}}$ and divide the distance by the ray length:

$$d_{\text{planar}}(u, v) = \frac{d_{\text{euclidean}}(u, v)}{\|\mathbf{r}(u, v)\|}, \tag{6}$$

where $\mathbf{r}(u, v) = [u_{\text{ndc}},\, v_{\text{ndc}},\, 1]^\top \cdot (M_{\text{cam}}^{\text{ndc}})^\top$ is the unnormalized ray direction for pixel (u, v) in NDC.

Invalid pixels (NaN, Inf, or non-positive depth) are masked out via a validity mask.

Additionally, the HDR images are tone-mapped to $[0, 1]$ using Reinhard tone mapping followed by gamma correction ($\gamma = 2.2$):

$$I_{\text{rgb}} = \left(\frac{I_{\text{hdr}}}{1 + I_{\text{hdr}}} \right)^{1/\gamma}. \tag{7}$$

2.2 Models Under Evaluation

We compare three depth estimation configurations, all implemented in the `ppdr` codebase:

1. **PPD** — Pixel-Perfect Depth (ViT-L backbone, 4 sampling steps, 1024×768 target area). Uses Depth Anything v2's ViT-L encoder as the semantics prompter. Predictions are relative log-depth. To compute meaningful metrics, we align them to metric depth via affine alignment in log space using the ground truth depth from Hypersim (see §2.4).
2. **DAv2** — Depth Anything v2 (ViT-L, standalone). Outputs relative disparity; aligned to metric depth via affine alignment in disparity space.
3. **DAv2-Cleaned** — Depth Anything v2 with post-hoc flying-pixel cleaning (see §2.3). Identical inference to DAv2, followed by the cleaning heuristic; then aligned.

All models are run on an NVIDIA GPU with CUDA; warm-up batches are used to avoid cold-start timing artifacts.

2.3 Flying-Pixel Cleaning Heuristic

The heuristic implemented in `ppdr.utils.heuristic.clean_flying_pixels` targets pixels that exhibit a strong depth gradient but *no corresponding edge in the RGB image*. The intuition is that a true object boundary should be visible in both the depth map *and* the color image. A strong depth discontinuity that does not coincide with an RGB edge likely represents a flying-pixel artifact.

Algorithm 1 Flying-Pixel Cleaning Heuristic

Require: Depth map $D \in \mathbb{R}^{B\times H\times W}$, RGB image $I \in \mathbb{R}^{B\times 3\times H\times W}$
Ensure: Cleaned depth map $D' \in \mathbb{R}^{B\times H\times W}$

```
Normalize D per image: D_n ← (D − D_min)/(D_max − D_min)
Compute depth gradient via Sobel filter: G ← Sobel(D_n)
Normalize gradient: G_n ← G/G_max
Extract RGB edges via Canny (on luma channel): thresholds (0.05, 0.10)
Flying mask: M_fly ← (G_n > τ_depth) ∧ (RGB_edges = 0)
Dilate M_fly with a 3 × 3 kernel
D' ← D · ¬M_fly                                   ▷ Zero out flying pixels
```

The default threshold $\tau_{\text{depth}} = 0.05$ controls the sensitivity: lower values remove more pixels. The dilation step ensures that the full "halo" around each flying pixel is removed.

The key operations (Sobel, Canny, morphological dilation) are performed using Kornia [3] on GPU, making the heuristic fast (approximately 80 ms overhead per image).

2.4 Depth Alignment for Metric Recovery

Since both PPD and DAv2 produce relative depth predictions, we align them to the ground-truth metric depth before computing F-scores.

DAv2 (disparity space alignment). DAv2 outputs relative disparity $\hat{d}_{\text{disp}}$. We solve for affine parameters (a, b) minimizing:

$$\min_{a,b} \sum_{i\in\text{valid}} \left(a\,\hat{d}_{\text{disp},i} + b - 1/d_{\text{gt},i}\right)^2, \tag{8}$$

and then recover metric depth as $d_{\text{metric}} = 1/(a\,\hat{d}_{\text{disp}} + b)$, clamped to $[10^{-3},\, d_{\max}]$.

PPD (log-depth space alignment). PPD outputs predictions in log-depth space. We solve:

$$\min_{a,b} \sum_{i\in\text{valid}} \left(a\,\hat{d}_{\text{ppd},i} + b - \log(d_{\text{gt},i} + \epsilon)\right)^2, \tag{9}$$

with $\epsilon = 1$, and then recover $d_{\text{metric}} = \exp(a\,\hat{d}_{\text{ppd}} + b) - \epsilon$.

Both systems are solved via normal equations $(A^\top A)\mathbf{x} = A^\top \mathbf{b}$.

3 Evaluation Metrics

3.1 Depth F-score ($\delta = 1.25$)

The F-score measures the fraction of predicted pixels whose depth ratio with the ground truth is below a threshold δ:

$$\text{accurate}(i) = \mathbb{1}\left[\max\left(\frac{d_{\text{pred},i}}{d_{\text{gt},i}},\, \frac{d_{\text{gt},i}}{d_{\text{pred},i}}\right) < \delta\right]. \tag{10}$$

Precision is the fraction of predicted valid pixels that are accurate. **Recall** is the number of accurate pixels divided by the total number of ground-truth valid pixels. The **F-score** is the harmonic mean of precision and recall.

For DAv2-Cleaned, precision is computed only over the surviving (non-zeroed) pixels. If the heuristic removes a pixel, it does not count against precision, but it *does* reduce recall. This captures the fundamental trade-off of the heuristic: removing flying pixels improves local accuracy (precision) but loses information (recall).

3.2 Edge-Aware Chamfer Distance

To specifically assess edge quality, we compute an **edge-aware Chamfer distance** between two point clouds sampled at edge regions:

1. **GT edges:** extracted from the RGB image via the Canny operator (thresholds $\sigma_{\text{low}} = 0.2$, $\sigma_{\text{high}} = 0.5$) with dilation radius 1 px. These represent where depth discontinuities *should* occur.
2. **Predicted edges:** extracted from the predicted depth map via Canny on the normalized depth (thresholds 0.2 / 0.5, dilation 1 px). These represent where depth discontinuities *actually* occur.
3. Both edge-masked point clouds are normalized by the median ground-truth depth for scale invariance.
4. The **Chamfer distance** is computed as the average of the two directed nearest-neighbor distances using KD-trees:

$$\mathrm{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\| + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|. \tag{11}$$

The Canny operator applied to the RGB image for computing the edge mask of the reference point cloud uses the following parameters: `canny_low`= 0.2, `canny_high`= 0.5, `dilation_px`= 1. Figure 1 shows an example of the resulting edge mask superimposed on the original image.

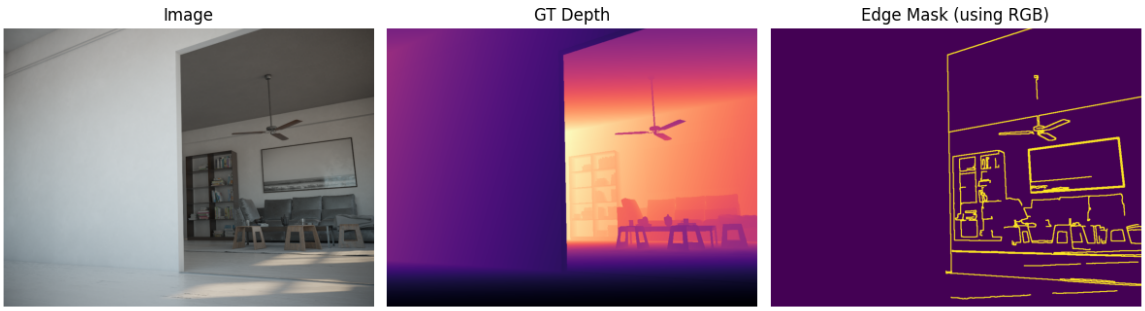


Figure 1: Example edge mask obtained via Canny edge detection on the RGB image (`canny_low`= 0.2, `canny_high`= 0.5, `dilation_px`= 1) on a sample from the Hypersim test set.

A *lower* Chamfer distance indicates that the model places depth discontinuities at the correct spatial locations (i.e. aligned with true RGB edges), which is the hallmark of flying-pixel-free depth estimation.

3.3 Inference Time

We measure per-image inference time (in milliseconds) after GPU warm-up, using CUDA synchronization barriers for accurate timing. We used a GPU L4 and an inference batch size of 2.

4 Benchmark Pipeline

The full benchmarking pipeline, implemented in the `benchmark.py` script and the benchmark utility module, proceeds as follows:

1. **Data loading.** The `HypersimReader` loads HDR images (tone-mapped to $[0, 1]$ RGB), ground-truth distances (converted to planar depth), and camera intrinsics for each test entry.
2. **Model warm-up.** Each model is invoked on one warm-up batch to stabilize GPU kernel caching, memory allocation, and JIT compilation.
3. **Timed inference.** For each batch, the model's `predict()` method is called with CUDA synchronization barriers before and after to measure wall-clock inference time.
   - PPD: resizes to a target area of 1024×768, runs 4 diffusion steps under mixed-precision autocast, and resizes back.
   - DAv2: resizes to 518×518, runs a single forward pass, and resizes back.
   - DAv2-Cleaned: runs DAv2, then applies the flying-pixel cleaning heuristic.

4. **Metric depth alignment.** The relative predictions are aligned to the ground-truth metric depth (see §2.4).
5. **Metric computation.**
   - Edge-aware Chamfer distance (§3.2)
   - Depth F-score with $\delta = 1.25$ (§3)
   - Per-image inference time
6. **Aggregation.** Results are stored per-image in a JSON file and summarized as mean ± standard deviation across the test set.

5 Results

5.1 Quantitative Results

Tables 1 to 3 summarize the benchmarking results across 100 Hypersim test images.

Table 1: Depth F-score metrics ($\delta = 1.25$). Mean ± standard deviation across the test set.

Model	Precision	Recall	F-score
PPD	0.656 ± 0.361	0.656 ± 0.361	0.656 ± 0.361
DAv2	0.655 ± 0.338	0.644 ± 0.333	0.650 ± 0.335
DAv2-Cleaned	0.649 ± 0.336	0.608 ± 0.320	0.628 ± 0.327

Table 2: Per-image inference time (milliseconds), measured on GPU after warm-up.

Model	Inference Time (ms)
PPD	2317.5 ± 11.1
DAv2	290.2 ± 2.9
DAv2-Cleaned	369.0 ± 38.6

Table 3: Edge-aware Chamfer distance (lower is better). NaN values are excluded.

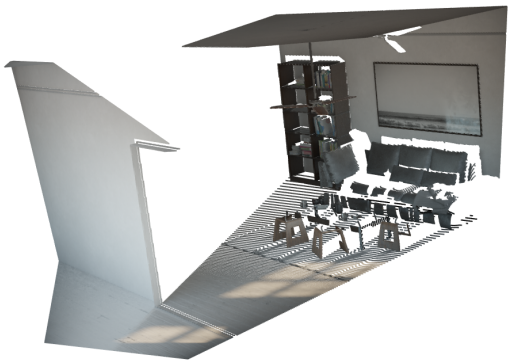
Model	Chamfer Distance
PPD	0.469 ± 0.450
DAv2	0.619 ± 0.828
DAv2-Cleaned	$\mathbf{0.394 \pm 0.332}$

Several observations emerge:

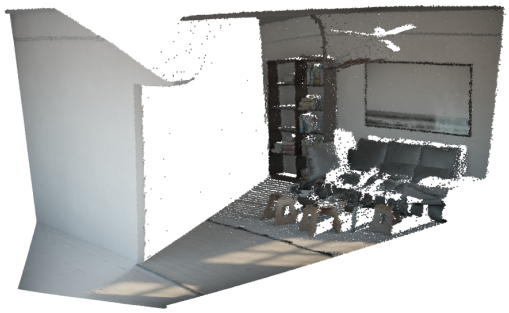
- **Edge quality.** DAv2-Cleaned achieves the best edge-aware Chamfer distance, outperforming both DAv2 and PPD. The heuristic effectively removes flying pixels at edges, bringing the predicted depth discontinuities closer to the true RGB boundaries.
- **Depth accuracy.** PPD achieves the highest F-score (0.656), closely followed by DAv2 (0.649). DAv2-Cleaned has a lower F-score (0.627) because zeroing out pixels reduces recall: removed pixels no longer count as accurate even if some of them were actually correct.
- **Speed.** DAv2 is approximately 8× faster than PPD (290 vs 2318 ms). The cleaning heuristic adds only ∼80 ms overhead, making DAv2-Cleaned still roughly 6.5× faster than PPD.

5.2 Qualitative Results: 3D Point Clouds

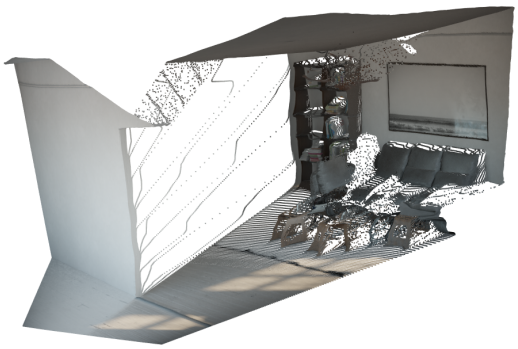
Figure 2 shows the 3D point clouds reconstructed from depth predictions on a representative Hypersim scene (`ai_048_010_cam_03_47`).



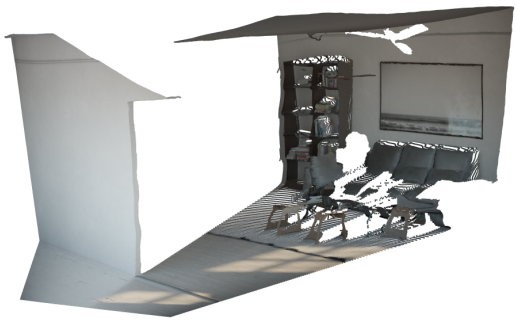
(a) Ground-truth point cloud.



(b) PPD point cloud.



(c) DAv2 point cloud — the flying pixels are visible as "haloes" around object and wall boundaries.



(d) DAv2-Cleaned point cloud — the heuristic removes most flying pixels, at the cost of removing some valid edge points.

Figure 2: 3D point cloud comparison on Hypersim scene `ai_048_010_cam_03_47`. The ground-truth (a) is the reference. PPD (b) produces clean edges. DAv2 (c) exhibits visible flying pixels. DAv2-Cleaned (d) reduces these artifacts via post-hoc filtering.

The ground-truth point cloud (fig. 2a) shows clean, sharp edges between objects. The PPD reconstruction (fig. 2b) is qualitatively close to the ground truth at edges. The DAv2 reconstruction (fig. 2c) shows clearly visible spurious points between the foreground and background surfaces at object boundaries. After applying the heuristic (fig. 2d), most of these flying points disappear, though some valid edge points are also removed, resulting in slightly sparser boundaries.

6 Discussion

6.1 Trade-offs of the Heuristic Approach

The heuristic delivers a remarkably effective edge-quality improvement at minimal computational cost. However, it comes with several trade-offs:

Information loss. The heuristic operates by *removing* pixels. Zeroed-out pixels reduce the recall component of the F-score, meaning the cleaned point cloud is sparser. In applications where point density is important (e.g. dense 3D reconstruction), this loss may be unacceptable.

Sensitivity to thresholds. The heuristic depends on hyperparameters (depth gradient threshold $\tau_{\text{depth}} = 0.05$, Canny thresholds, dilation kernel size). These were tuned on a few examples and may not generalize optimally across all scene types.

Failure on highly textured scenes. For images containing fine textures (most notably **vegetation**, see figure 3), the Canny edge detector on the RGB image produces dense edge maps, causing the flying-pixel mask to become very conservative (few pixels are flagged as flying). As a result, the heuristic does not remove flying pixels in regions with visually complex textures such as leaves, branches, and other organic structures. PPD, by contrast, handles these cases gracefully since its pixel-space diffusion naturally preserves sharp depth transitions at fine structures.

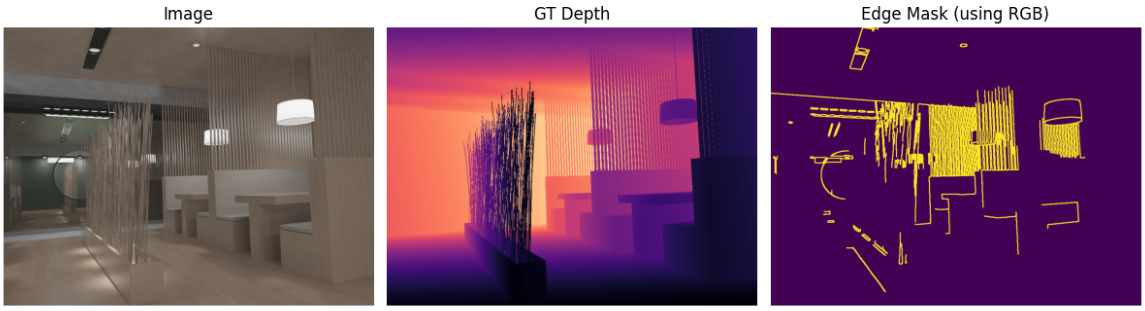


Figure 3: Example of badly recovered edge mask with the Canny operator.

6.2 End-to-End vs. Post-Hoc

While the heuristic achieves competitive (and even superior) edge-aware Chamfer distances, it may be considered a less *satisfying* solution than PPD's end-to-end approach:

- PPD learns to *not produce* flying pixels in the first place, by modelling pixel-wise depth distributions in pixel space.
- The heuristic is a hand-crafted rule that *removes* artifacts after the fact, with no guarantee of optimality.
- PPD's approach is fundamentally more principled: the model directly addresses the root cause (VAE compression / regression averaging), whereas the heuristic treats the symptoms.

7 Conclusion

In this project, we investigated the central claim of Pixel-Perfect Depth: that pixel-space diffusion produces depth maps with significantly fewer flying pixels at object boundaries compared to prior methods.

Rather than reproducing the paper's full benchmark, we tested whether a *simpler approach*, a post-hoc heuristic applied to Depth Anything v2's predictions, could approximate PPD's edge-quality benefits.

Our key findings are:

1. **The heuristic is effective.** DAv2-Cleaned achieves the best edge-aware Chamfer distance among all three models, outperforming both raw DAv2 and PPD.
2. **Speed advantage.** DAv2-Cleaned is approximately 6.5× faster than PPD (369 vs 2318 ms), making it suitable for real-time or latency-sensitive applications.
3. **Information trade-off.** The heuristic reduces recall (F-score drops from 0.649 to 0.628) because it removes pixels, leading to sparser point clouds.
4. **Texture sensitivity.** The heuristic performs poorly on highly textured regions (e.g. vegetation), where dense RGB edges prevent the flying-pixel detector from flagging artifacts. PPD handles such cases significantly better.
5. **Principled vs. pragmatic.** PPD's end-to-end, pixel-space diffusion approach is more principled and eliminates the root cause of flying pixels. The heuristic is a pragmatic workaround that treats symptoms, not causes, and cannot match PPD's quality in all scenarios.

In summary, the post-hoc heuristic is a simple and efficient alternative for applications where edge quality matters and scene complexity is moderate. For challenging scenes with fine structures, an end-to-end solution like PPD remains the superior choice.

References

[1] Aleksei Bochkovskii et al. "Depth pro: Sharp monocular metric depth in less than a second". In: *arXiv preprint arXiv:2410.02073* (2024).

[2] Bingxin Ke et al. "Repurposing diffusion-based image generators for monocular depth estimation". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2024, pp. 9492–9502.

[3] E. Riba et al. "Kornia: an Open Source Differentiable Computer Vision Library for PyTorch". In: *Winter Conference on Applications of Computer Vision*. 2020. URL: https://arxiv.org/pdf/1910.02190.pdf.

[4] Mike Roberts et al. *Hypersim: A Photorealistic Synthetic Dataset for Holistic Indoor Scene Understanding*. Aug. 2021. DOI: 10.48550/arXiv.2011.02523. arXiv: 2011.02523 [cs]. (Visited on 04/04/2026).

[5] Gangwei Xu et al. *Pixel-Perfect Depth with Semantics-Prompted Diffusion Transformers*. Oct. 2025. DOI: 10.48550/arXiv.2510.07316. arXiv: 2510.07316 [cs]. (Visited on 03/12/2026).

[6] Lihe Yang et al. "Depth anything v2". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 21875–21911.